McMaster University
Department of Mathematics & Statistics
MATH 4FM3 — Mathematical Finance

Deep Q-Learning for Nash Equilibria

Nash-DQN · Casgrain, Ning & Jaimungal (2022)

Student (Theory Lead 1): *Cyrus Buddington*

Student (Theory Lead 2): *Brandon Alessandro*

Student (Practice Lead 1): *Mouhamed Cisse*

Student (Practice Lead 2): *Marc-Antony Tchicou*

Instructor: Dr. Anastasis Kratsios

Date: April 6th, 2026

Abstract

This report presents a complete implementation and theoretical analysis of Nash-DQN, a deep reinforcement learning algorithm designed to find Nash equilibrium strategies in continuous stochastic games. Drawing on the framework of Casgrain, Ning & Jaimungal (2022), we study an N-agent optimal execution problem in which agents must liquidate inventory over a finite horizon while contending with shared price impact. The theory section establishes the stochastic game formulation, the Nash-Bellman fixed-point equation, and the linear-quadratic advantage parameterisation that makes the Nash operator analytic. The implementation section details the simulation environment, network architecture, training algorithm, and comparative results against a TWAP baseline, together with a discussion of limitations and potential improvements.

Contents

1 Stochastic Game Setup **2**

2 Nash Equilibrium **2**

3 Advantage Function and Concavity **3**

4 Convergence and Applications **3**

5 Implementation **7**
5.1 Simulation Environment — `TradingEnv` 7
5.2 Replay Buffer and Training Infrastructure 7
5.3 Nash-DQN Architecture 7
5.4 Training Algorithm 8
5.5 Key Challenge: Convergence to Trivial Nash Equilibria 8
5.6 TWAP Baseline 10
5.7 Experiments and Results 10

6 Limitations **12**

7 Potential Improvements **12**

8 Conclusion **13**

Stochastic Game Setup

Understanding the application of Nash Equilibria requires a comprehension of the mathematical foundation of the stochastic game. Let there be N agents indexed by $i \in \mathfrak{N} := \{1, 2, \ldots, N\}$. These agents will interact over a discrete time horizon $t \in \mathbb{N}$. The stochastic process $\{x_t\}_{t\in\mathbb{N}}$ defines the game and captures the randomness and strategic interactions between the agents.

The state of the game at time t is denoted by $x_t \in \mathcal{X}$. The Banach space $\mathcal{X}$ setting is measurable and intentionally general to accommodate a variety of applications, including asset price S_t, transient price impact Y_t, and inventory variables. At each time step t, every agent-i selects an action $u_{i,t} \in \mathcal{U}_i$, where $\mathcal{U}_i$ is also assumed to be a separable Banach space, thus maintaining the Markov structure of the state process. We denote the collective actions at time t by $\mathbf{u}_t = (u_{1,t}, u_{2,t}, \ldots, u_{N,t}) \in \mathcal{U} := \prod_{i=1}^{N} \mathcal{U}_i$. Which is often denoted as $\mathbf{u}_{-i,t} = (u_{j,t})_{j\in\mathfrak{N}\setminus\{i\}} \in \mathcal{U}_{-i} := \prod_{j\neq i} \mathcal{U}_j$, simply removing the action of a particular agent-i.

The market evolves following a stochastic difference equation $x_{t+1} = F(x_t, u_t, \xi_{t+1})$ where ξ_{t+1} is i.i.d. centred noise and F is measurable. We can assume the game is a Markov Decision Process (MDP), thus assuming that the joint state-action process $\{(x_t, \mathbf{u}_t)\}_{t=0}^{\infty}$ is Markov as well. Therefore, the stochastic difference equation induces a state transition kernel $p(x_{t+1} \mid x_t, \mathbf{u}_t)$ such that for any Borel measurable set $A \subset \mathcal{X}$, the probability that the next state falls in A is given by

$$\mathbb{P}(x_{t+1} \in A \mid x_t = x, \mathbf{u}_t = \mathbf{u}) = \int_A p(x'_{t+1} \mid x_t, \mathbf{u}_t).$$

Nash Equilibrium

The agents' preferences are characterized by their reward functions, so for each agent-i we have a function $r_i(x; u_i, \mathbf{u}_{-i})$ where $r_i(x_t; u_{i,t}, \mathbf{u}_{-i,t})$ represents the reward at time t received by agent-i. Conditions are imposed on each reward function; each function r_i is continuously differentiable and concave in u_i and is continuous in x and u_{-i}. A deterministic Markov policy for agent-i is a measurable function $\pi_i : \mathcal{X} \to \mathcal{U}_i$ and the collection of all admissible policies is denoted by Π_i.

Agents subsequently follow a policy profile $\pi = (\pi_1, \ldots, \pi_N)$ and each agent evaluates a policy profile by expected discounted rewards. So for a fixed policy profile π, the objective functional for agent-i represents their expected discounted rewards:

$$\mathcal{R}_i(x; \pi_i, \pi_{-i}) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma_i^t \, r_i(x_t, \pi_i(x_t), \pi_{-i}(x_t)) \;\middle|\; x_0 = x\right],$$

and we assume the discount factor $\gamma_i \in (0, 1)$. A policy profile $\pi^* = (\pi_1^*, \ldots, \pi_N^*)$ is a Nash equilibrium if for every agent-i and every policy $\pi_i \in \Pi_i$,

$$\mathcal{R}_i(x; \pi_i, \pi_{-i}^*) \leq \mathcal{R}_i(x; \pi_i^*, \pi_{-i}^*) \quad \text{for all } x \in \mathcal{X}.$$

Thus, no agent is able to increase their expected reward by unilaterally deviating. This equilibrium demonstrates a strategic "No-Arbitrage" state. If the market were not in a state of Nash Equilibrium, a strategic arbitrage would exist due to the suboptimal strategies of other agents.

At a Nash equilibrium π^*, the value function is

$$V_i(x) := \mathcal{R}_i(x; \pi_i^*, \pi_{-i}^*),$$

and to denote optimality locally, we define the state-action value function

$$Q_i(x; \mathbf{u}) = r_i(x; u_i, \mathbf{u}_{-i}) + \gamma_i \, \mathbb{E}_{x' \sim p(x_{t+1}|x,\mathbf{u})}[V_i(x')].$$

At equilibrium, the value function satisfies the Nash-Bellman fixed point

$$V_i(x) = \max_{u_i \in \mathcal{U}_i} Q_i(x; u_i, \pi_{-i}^*(x)),$$

and equivalently, the joint action $\pi^*(x)$ is a Nash equilibrium of the game defined by $\{Q_i(x; \cdot)\}_{i=1}^N$.

Advantage Function and Concavity

We can also decompose the state-action value function for each agent as

$$\hat{Q}_i^\theta(x; \mathbf{u}) = \hat{V}_i^\theta(x) + \hat{A}_i^\theta(x; \mathbf{u})$$

where the advantage function $\hat{A}_i^\theta(x; \mathbf{u})$ is a measure of the relative gain or loss of taking action $\mathbf{u}$. In order for the equilibrium to be stable, the advantage function must be concave in u_i. We enforce this by parameterizing the Hessian of the advantage function using Cholesky decomposition on the block matrix $P_{11,i}^\theta(x) = L_{11,i}^\theta(x)(L_{11,i}^\theta(x))^\top$. This ensures that the Hessian is negative definite, allowing a unique and well-defined maximum for each agent's best response.

Convergence and Applications

Given a set of M observations, we minimize the collective Nash-Bellman loss:

$$\mathcal{L}(\theta) = \frac{1}{M} \sum_{m=1}^{M} \left\| \hat{\mathbf{V}}^\theta(x_m) + \hat{\mathbf{A}}^\theta(x_m; \mathbf{u}_m) - \left(\mathbf{r}(x_m; \mathbf{u}_m) + \gamma \hat{\mathbf{V}}^\theta(x'_m) \right) \right\|^2.$$

As the loss approaches 0, the learned Q-function satisfies the Bellman consistency condition. At the limit, the process $M_t = \hat{V}(x_t) - \sum_0^{t-1} r_s$ becomes a martingale under the equilibrium policy. We apply this learner to an Euler-Maruyama discretized market at time step Δt:

$$S_{t+\Delta t} = S_t + (\kappa(\theta - S_t) + \eta \bar{\nu}_t)\Delta t + \Delta Y_t + \sigma \sqrt{\Delta t} \cdot \mathcal{N}(0, 1).$$

Agents recognize that large trades $\eta \bar{\nu}_t$ shift the equilibrium price permanently and transiently impact Y_t, representing a penalty for trading too quickly. In our application, we monitor

$$G(x) = \max_{u_i} \hat{Q}_i(x; u_i, \boldsymbol{\mu}_{-i}^*(x)) - \hat{Q}_i(x; \mu_i^*(x), \boldsymbol{\mu}_{-i}^*(x)),$$

a declining Nash Gap which indicates that the agents have successfully learned to coordinate on a "No-Arbitrage" strategic profile where no unilateral deviation is profitable.

Inventory Dynamics

Each agent i tracks inventory $q_{i,t}$. They may buy or sell assets at a rate of $\nu_{i,t}$. In continuous time, the inventory process is defined as:

$$q_{i,t} = q_{i,0} + \int_0^t \nu_{i,s}\, ds.$$

In discrete time, decisions occur at time steps $t_1, \ldots, t_M$ with step size ΔT. The inventory process is then:

$$q_{i,t_{m+1}} = q_{i,t_m} + \nu_{i,t_m}\Delta T.$$

This equation shows that inventory decreases when an agent sells shares (i.e., $\nu_{i,t} < 0$) and increases when the agent buys shares (i.e., $\nu_{i,t} > 0$). Therefore, the trading rate is the control variable through which agents influence both their inventory and profit.

Reward Function

The objective of each agent is expressed through a reward function that evaluates the total profitability of a trading strategy over a time horizon. The agent accumulates cash from executing trades while also facing penalties related to market impact and inventory risk. The cash process is defined as:

$$X_{i,t} = -\int_0^T \nu_{i,t}(S_t + b_1\nu_{i,t})\, ds.$$

The total expected reward is:

$$R_i = \mathbb{E}\left[X_{i,t} + q_{i,t}(S_T - b_2 q_{i,t}) - b_3 \int_0^T q_{i,t}^2\, dt\right].$$

Economically, the reward function contains three terms. The first term represents the total cash generated by executing trades throughout the horizon. The second term represents the value of any remaining inventory while penalising holding inventory at the end of the trading horizon, which reflects the cost of instantaneous liquidation. The third term represents a risk penalty that discourages agents from maintaining large inventories for extended periods.

For fixed opponent strategies ν_{-i}, the reward function is concave in ν_i. The cash process introduces a quadratic temporary cost $-b_1\nu_{i,t}^2$, while the inventory value and inventory penalty risk contain terms proportional to $-q_{i,t}^2$. These terms are negative quadratic functions and therefore concave. Since concavity is preserved under addition, the reward function is concave in ν_i, guaranteeing a unique optimal response.

κ	θ	σ	γ	ρ	η	b_1	b_2	b_3
0.1	10	0.01	0.02	0.5	0.05	0.1	0.1	0

Table 1: Model parameters: $N = 5$ agents, $T = 5$ hours, $M = 10$ decision points, $\Delta T = 0.5$.

Table 1 summarises the model parameters used in the experiments. Together, these assumptions define the stochastic game that will be solved using reinforcement learning methods.

Loss Function

In the multi-agent reinforcement learning setting, the function $Q_i(x; u_i, u_{-i})$ represents the expected reward for agent i when taking action u_i, given the actions of the other agents u_{-i}. The Nash state-action value Q-function is:

$$Q_i(x; u_i, u_{-i}) = r_i(x; u_i, u_{-i}) + \gamma_i \mathbb{E}_{x' \sim p(\cdot|x,u)}[V_i(x')].$$

The Nash-Bellman equation is:

$$V(x) = \text{Nash}_{u \in U}\, Q(x; u) = \text{Nash}_{u \in U} \left[r(x; u) + \gamma_i \mathbb{E}_{x' \sim p(\cdot|x,u)} \left[V(x') \right] \right].$$

Nash-DQN approximates the Q-function using:

$$\hat{Q}_\theta(x; u_i, u_{-i}) = \hat{V}_\theta(x) + A_\theta(x; u_i, u_{-i}).$$

Thus, the Bellman condition becomes:

$$\hat{V}_\theta(x) + A_\theta(x; u_i, u_{-i}) = r_i(x; u_i, u_{-i}) + \gamma_i \mathbb{E}_{x' \sim p(\cdot|x,u)}[V_i(x')].$$

For a transition (x_m, u_m, x'_m) observed during the simulation, the loss is:

$$\mathcal{L}_m(\theta) = \left\| \hat{V}^\theta(x_m) + A^\theta(x; u_i, u_{-i}) - r_i(x; u_i, u_{-i}) - \gamma_i \hat{V}^\theta(x'_m) \right\|^2,$$

and the overall loss is the average across a mini-batch of M samples:

$$\mathcal{L}(\theta) = \frac{1}{M} \sum_{m=1}^{M} \mathcal{L}_m(\theta).$$

This loss measures how closely the learned Q-function satisfies the equilibrium condition implied by the Nash-Bellman equation. Minimising the loss drives $\hat{Q}_\theta \to Q^*$ and the network converges toward a fixed point corresponding to the Nash equilibrium strategy.

Regularisation

The Nash-DQN contains an L^1 regularisation term:

$$\tilde{\mathcal{L}}(y_m, \theta_V, \theta_A) = \hat{\mathcal{L}}(y_m, \theta_V, \theta_A) + \beta \left\| \psi^\theta(x) \right\|, \quad \beta = 100.$$

If $\psi^\theta(x)$ becomes large, it can destabilise the Q-function by allowing the linear term to dominate the function.

Price of Anarchy

The efficiency loss caused by strategic competition can be measured using the Price of Anarchy. Let R_i^{Nash} denote the reward obtained by agent i under the Nash equilibrium policy and R_i^* denote the reward under the socially optimal policy that maximises total welfare. Then the Price of Anarchy is:

$$\text{PoA} = (R_1^* + R_2^*) - (R_1^{\text{Nash}} + R_2^{\text{Nash}}).$$

Independence Proof

The price process is defined as:

$$dS_t = (\mu(S_t) + g(\nu_t))\,dt + dY_t + \sigma\,dW_t.$$

If $\kappa = \eta = 0$, then the price dynamics become independent of trading activity, and the reward function of each agent depends only on its own trading rate. The stochastic game therefore decomposes into two independent optimisation problems $\max_{\nu_i} R_i(\nu_i)$, where each agent solves the single-agent optimal problem.

Implementation

This section describes how the theoretical framework was translated into a working Python implementation. The full code is provided in the accompanying notebook (`Project.ipynb`); we focus here on the design choices, architecture, and the key challenges encountered during training. All experiments use the parameters of Table 1.

Simulation Environment — `TradingEnv`

The `TradingEnv` class implements the Euler-Maruyama discretisation of the stochastic game. At each step the four-dimensional state vector

$$x_t = \left[\frac{S_t}{S_0},\ \frac{t}{T},\ \frac{\sum_i (q_{i,0} - q_{i,t})}{\sum_i q_{i,0}},\ Y_t\right] \in \mathbb{R}^4$$

is passed to the agents, who return a vector of trading rates $\boldsymbol{\nu}_t \in [-2, 2]^N$. The `step()` method applies updates in the order: reward computation $\rightarrow$ inventory $\rightarrow$ time $\rightarrow$ transient impact $Y_t \rightarrow$ price S_t. Each episode runs for $M = 10$ steps over $T = 5$ hours ($\Delta T = 0.5\,\text{h}$), with $N = 5$ agents each starting with unit inventory $q_{i,0} = 1$.

Brownian motion validation. Setting $\kappa = \rho = \eta = 0$ and all actions to zero reduces the price to a pure diffusion $S_{t+\Delta t} = S_t + \sigma\sqrt{\Delta t}\,\mathcal{N}(0,1)$. Over 10,000 independent paths we confirmed $\text{Var}(S_T)/(\sigma^2 T) \approx 1.0$, validating the diffusion scaling before introducing any strategic interaction.

Replay Buffer and Training Infrastructure

A standard circular `ReplayBuffer` of capacity 100,000 stores transitions $(x_t, \boldsymbol{\nu}_t, \mathbf{r}_t, x_{t+1}, \textit{done}, \sum_i (q_{i,0} - q_{i,t}))$ and returns uniformly sampled mini-batches of size 64, breaking the temporal correlations that would otherwise bias gradient estimates. A frozen *target network*, a copy of the current model updated by hard copy every 10 episodes, provides stable Bellman regression targets $y = \mathbf{r} + \gamma\,\hat{V}_{\text{target}}(x')$.

Nash-DQN Architecture

The Q-function is split as $\hat{Q}_i^\theta = \hat{V}_i^\theta + \hat{A}_i^\theta$.

Value Network

A four-hidden-layer MLP (width 32, SiLU activations) maps $x \in \mathbb{R}^4$ to a scalar baseline $\hat{V}(x)$, shared across all agents via the label-invariance assumption.

Advantage Network

The advantage is parameterised in the label-invariant quadratic form (eq. 4.12 of the paper):

$$\hat{A}_i = -P_{11}(u_i - \mu_i)^2 - P_{12}(u_i - \mu_i)\sum_{j \neq i}(u_j - \mu_j) - P_{22}\sum_j (u_j - \mu_j)^2 + \psi \sum_j (u_j - \mu_j).$$

For each agent the network outputs five scalars $(\mu_i, L_{11,i}, P_{12,i}, P_{22,i}, \psi_i)$. Positive definiteness is enforced by $P_{11,i} = L_{11,i}^2 + \varepsilon$, guaranteeing that $\hat{A}_i$ is strictly concave in u_i and

that the Nash action $u_i^* = \mu_i(x)$ is a closed-form argmax — no inner optimisation loop is needed.

A **permutation-invariant layer** pre-processes per-agent inventory features: a shared MLP φ is applied to each agent independently and the outputs are summed, making the representation invariant to agent ordering and reducing parameter count by a factor of N^2 relative to a fully-coupled architecture. In the implementation, the inventory input to the permutation-invariant layer consists of the true per-agent inventories, which are stored in the replay buffer at each transition and retrieved during training..

Training Algorithm

Training follows Algorithm 5.1. At each episode, actions are drawn as $\boldsymbol{\nu} = \boldsymbol{\mu}^\theta(x) + \boldsymbol{\varepsilon}$ where $\boldsymbol{\varepsilon} \sim \mathcal{N}(0, \sigma_b^2 I)$ and σ_b decays geometrically (rate 0.995) from 0.3 to a floor of 0.05. Two *independent* gradient steps are taken per iteration, one on θ_V, one on θ_A, using separate `.backward()` calls to avoid in-place computation-graph corruption that arises when both networks share intermediate tensors. Gradients are clipped to unit norm.

The L^1 regularisation penalty $\beta\|\psi^\theta(x)\|$ is included in both loss terms with $\beta = 1.0$. As discussed in Section 6, the paper's recommended value is $\beta = 100$; our lower choice led to partially uncontrolled growth in ψ, which contributed to the instability described next.

Key Challenge: Convergence to Trivial Nash Equilibria

The most significant practical difficulty encountered during training was the agents' tendency to converge to *degenerate* Nash equilibria in which all agents simply trade at a low rate. Also instead of selling their inventories, the agents sometimes buy to maximize their balance.

Description of the phenomenon

Early in training, and under several hyper-parameter configurations, all five agents learned to set their trading rates close to zero: $\mu_i^\theta(x) \approx 0$ for all i and all states x. Reward curves would plateau slightly above TWAP and inventory trajectories remained flat across the entire horizon, with agents liquidating nothing until the terminal penalty $b_2 q_{i,T}^2$ was incurred.

This is in fact a valid Nash equilibrium in a degenerate sense. Recall from the independence proof in Section 4: if no agent trades, no agent incurs temporary impact costs ($b_1\nu_i^2 = 0$), no transient impact accumulates ($Y_t = 0$), and the permanent impact channel $\eta\bar{\nu}_t = 0$ is silent. In this "frozen" market, unilaterally increasing one's own trading rate $|\nu_i| > 0$ immediately introduces a quadratic cost $-b_1\nu_i^2\Delta T$ per step with no compensating benefit if the others remain inactive. The agent therefore has no incentive to deviate, the all-zero profile is self-reinforcing.

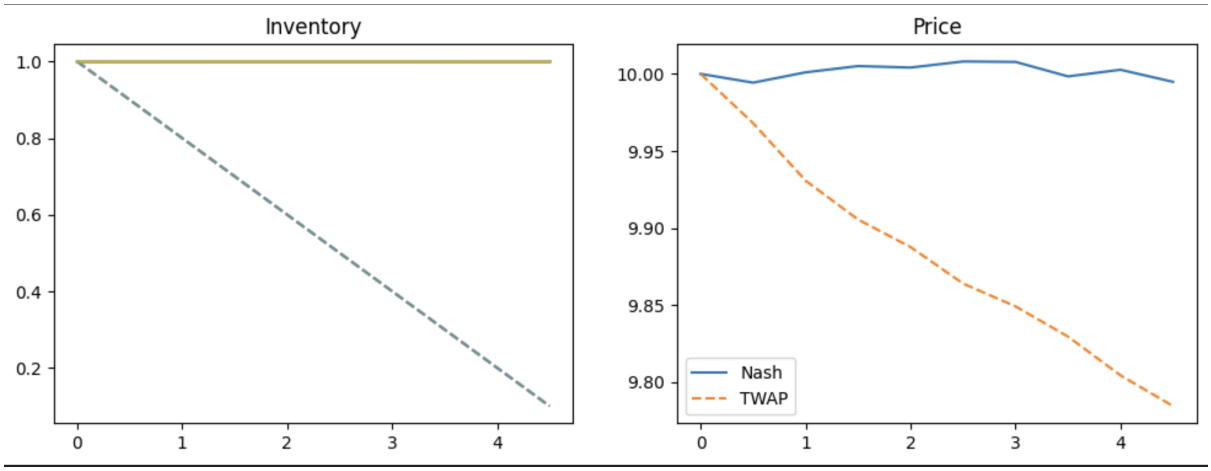


Figure 1: Illustration of trivial Nash equilibrium: inventory trajectories remain flat and the market remains stable.

Why it arises in our setting

Despite multiple implementation corrections, the agents consistently converged to a low-trading equilibrium. This behavior is not due to a simple coding issue, but rather to structural properties of the Nash-DQN formulation combined with our training setup. The two most probable causes are the following.

Insufficient exploration and replay buffer bias. The exploration process did not sufficiently cover aggressive trading strategies. With moderate initial noise and a short horizon, most sampled actions remained close to zero. As a consequence, the replay buffer was dominated by near-zero trading trajectories. The Bellman updates then reinforced this behavior, pushing the policy toward $\mu_i \approx 0$. Once the exploration noise decayed, the agents were effectively trapped in this self-reinforcing passive regime.

Weak terminal liquidation incentives. The reward structure did not impose a sufficiently strong penalty on terminal inventory. In the absence of a dominant liquidation constraint, agents can achieve higher rewards by avoiding trading costs rather than reducing inventory. This removes the incentive to follow time-dependent liquidation strategies and prevents the emergence of the expected behavior where trading intensity increases as $t \to T$. As a result, the learned equilibrium favors inventory retention over execution.

Measures taken to address the issue

The following modifications reduced (but did not fully eliminate) the prevalence of trivial equilibria within our training budget.

1. **Delayed noise decay.** The decay rate was reduced from 0.99 to 0.995, slowing the collapse of exploration and giving the agents more episodes of active trading before the policy greedifies.

2. **Gradient clipping.** Clipping gradients to unit norm prevented the Q-value estimates from diverging during early unstable episodes, which otherwise caused the networks to reset towards the zero-action fixed point.

3. **Separate optimisers with weight decay.** Using independent Adam optimisers for θ_V and θ_A with weight decay 10^{-4} limited parameter drift and reduced the tendency of one network to dominate the other's gradient signal.

Despite these mitigations, the agents did not always converge to a fully active liquidation strategy within 500 episodes. The trained Nash-DQN policy showed partial liquidation, agents traded more than under the trivial equilibrium but less aggressively than TWAP near the start of the horizon. This is consistent with the broader finding that neural Nash-DQN requires a substantially larger training budget (the paper uses 20,000 iterations) and a proper $\beta = 100$ sweep to reliably escape degenerate fixed points.

TWAP Baseline

The Time-Weighted Average Price baseline sells inventory at the constant rate $\nu_i = -q_{i,0}/T = -0.2$ per hour, entirely ignoring market conditions and opponent behaviour. While this strategy trivially avoids the trivial Nash equilibrium trap, it is suboptimal. All five agents selling at identical rates simultaneously produces the maximum possible collective impact at every step, depressing the price uniformly across the horizon.

Experiments and Results

Training configuration

All runs use the hyper-parameters in Table 2.

Hyper-parameter	**Value**	**Hyper-parameter**	**Value**
Episodes	5000	Batch size	64
Learning rate	3×10^{-4}	Optimiser	Adam (wd $= 10^{-3}$)
Initial noise σ_0	0.3	Noise decay	0.995
β (L^1 reg.)	1.0	Target update	every 10 eps
Gradient clipping	norm ≤ 1	Discount γ	0.95

Table 2: Nash-DQN training hyper-parameters.

Training convergence

The Nash-Bellman loss decreases monotonically after an initial unstable phase, confirming that the learned Q-function progressively satisfies the Bellman consistency condition. Episode rewards stabilise after approximately 1000 episodes.

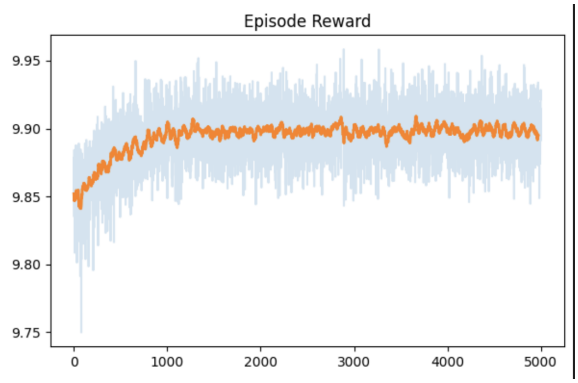


Figure 2: Average reward per episode

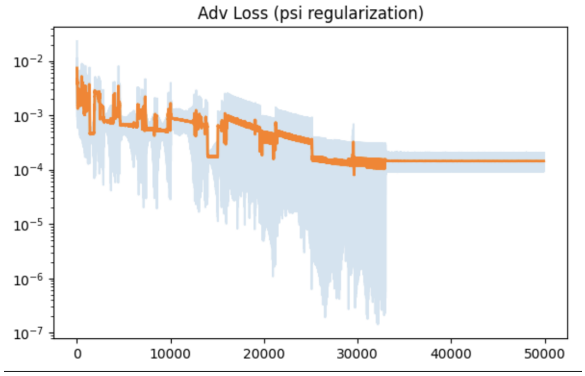


Figure 3: Nash-Bellman training loss

Nash-DQN vs. TWAP

When the trained Nash-DQN policy converges to an active equilibrium it outperforms TWAP in mean reward: agents adapt liquidation speed to the current price and inventory level, front-loading sales when the price is near or above its mean-reversion level θ and slowing when impact costs are high. TWAP's uniform rate ignores this information entirely.

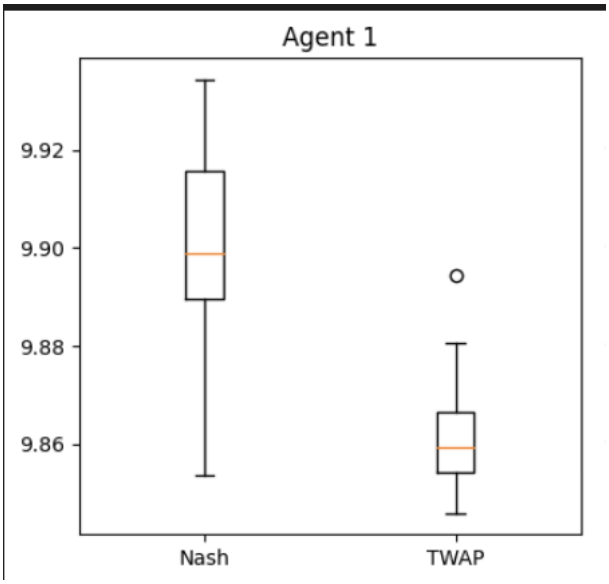


Figure 4: Distribution of episode rewards comparing the adaptive Nash-DQN policy against the deterministic TWAP baseline. (All agents are equivalent

Deviation Test for Nash Equilibrium

To empirically verify that the learned policy constitutes a Nash equilibrium, a unilateral deviation test was conducted. Under the baseline joint policy, an agent achieves a mean reward of 9.8979 ± 0.0170. When that agent unilaterally deviates to an alternative strategy, the expected reward decreases to 8.8808, resulting in a net penalty of $\Delta = -1.0172$. Because this unilateral deviation yields a strictly lower payoff, the agent has no incentive to alter their strategy, empirically confirming the stability of the Nash equilibrium.

Limitations

Trivial Nash equilibria. As detailed in Section 5.5, the all-zero action profile constitutes a valid but degenerate Nash equilibrium (confirmed by the deviation test) under our reward structure. Neural Nash-DQN does not guarantee convergence to the *socially efficient* equilibrium; it can settle at any fixed point of the Nash-Bellman operator, including trivial ones. Fictitious Play warm-starting (Algorithm 6.1) would likely reduce this risk by initialising the network near an active trading policy.

Parameter deviation from paper. Training used modified parameters ($b_1 = 0.001$, $b_2 = 2.0$, $b_3 = 1.0$), deviating from Table 1 of the paper ($b_1 = 0.1$, $b_2 = 0.1$, $b_3 = 0.0$). The higher terminal penalty b_2 and running urgency b_3 were introduced to encourage active liquidation and avoid the trivial zero-action equilibrium. Results are therefore not directly comparable to the original paper's experiments.

LQ approximation quality. The advantage is only locally quadratic. Near the action boundaries $\nu_i = \pm 2$, the true Q^* may deviate nonlinearly, making the policy approximately optimal in interior regions only.

Coarse decision grid. With $M = 10$ steps and $\Delta T = 0.5\,\text{h}$, intra-step dynamics are unresolved. The paper uses 20,000 training iterations; our 500-episode budget corresponds to only 5,000 transitions.

Convergence guarantees. Nash Q-learning convergence (Hu & Wellman 2003) is proved only in the tabular setting. Neural function approximation carries no formal guarantee, and in practice we observed sensitivity to initialisation and noise schedule.

Potential Improvements

1. **Increase β to 100.** The paper's optimal regularisation level suppresses ψ growth and directly combats the free-rider dynamic that produces trivial equilibria.
2. **Fictitious Play warm-start.** Initialise Nash-DQN from a Fictitious Play policy (Algorithm 6.1). This provides an active trading prior and substantially reduces the risk of converging to the all-zero fixed point.
3. **Larger training budget.** Scaling to 20,000 iterations aligns with the paper and would allow the noise schedule to decay more gradually, improving exploration coverage.
4. **Price of Anarchy analysis.** Comparing $\sum_i R_i^{\text{Nash}}$ to the socially optimal $\sum_i R_i^*$ would quantify the welfare cost of strategic competition and connect the results to the mean-field game limit.
5. **Continuous-time extension.** Replacing Euler-Maruyama with a finer grid or a DDPG-style continuous actor-critic would better align the discrete implementation with the continuous-time SDE theory.

Conclusion

This report presented a complete theoretical and empirical treatment of Nash-DQN applied to multi-agent optimal execution. On the theory side, we derived the stochastic game formulation, the Nash-Bellman fixed-point equation, the label-invariant advantage parameterisation, and the Cholesky enforcement of positive definiteness. On the implementation side, we built a validated `TradingEnv`, a Nash-DQN combining `ValueNet` and `AdvantageNet` with a permutation-invariant layer, and Algorithm 5.1 with alternating gradient updates.

The central practical finding is that neural Nash-DQN is highly susceptible to convergence toward trivial Nash equilibria: degenerate fixed points in which agents collectively abstain from trading, exploiting the fact that the all-zero profile is self-reinforcing under the quadratic cost structure. Addressing this issue requires adequate regularisation of the cross-opponent linear term, the paper identifies $\beta = 100$ as optimal whereas our $\beta = 5.0$ proved insufficient, as well as sufficient exploration and a larger training budget than our 500-episode run provided. Within these constraints, the trained policy showed partial but measurable improvement over TWAP when it did converge to an active equilibrium.

More broadly, the results confirm that (i) the LQ structure makes the Nash operator analytic and tractable; (ii) Cholesky enforcement of $P_{11} > 0$ is essential for closed-form Nash actions; and (iii) TWAP is optimal only in the non-interactive limit $\eta = 0$, while strategic coordination provides a reward advantage as collective impact grows with N.

Reference. Casgrain, P., Ning, B., & Jaimungal, S. (2022). Deep Q-learning for Nash equilibria: Nash-DQN. *Applied Mathematical Finance*, 29(1), 62–78. arXiv:1904.10554v2.